Experiment 5: Develop Activity and State Transition diagrams for the project using any open source tool

Part A — Activity Diagram

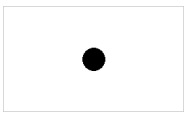
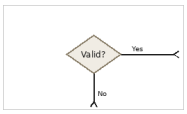
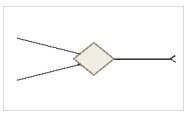
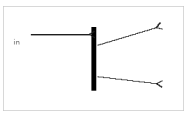
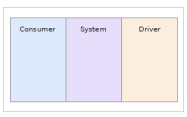
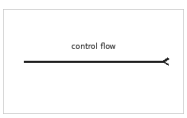
1. Description of the Diagram:

An Activity Diagram is a UML behavioural diagram that models the dynamic aspects of a system by showing the flow of control from one activity (action) to the next. It is the UML equivalent of a flowchart, extended with support for decisions, concurrency (fork/join), and swim lanes that show which actor is responsible for each step.

For this project, the activity diagram captures the complete, end-to-end asset lifecycle described in the SRS: a consumer or corporate IT manager submits an e-waste asset and (if required) flags a data-wipe request; the system fetches live metal prices and returns a quote; once accepted, the system schedules a pickup; the logistics driver verifies and photographs the packaging (FR-07) before transit; the recycle plant worker scans the item in, wipes data if flagged, and processes the recycling; the system then concurrently updates the asset status and generates a Certificate of Destruction (FR-10); and finally the certificate is emailed to the consumer while the compliance data is aggregated for the government regulator's read-only dashboard (FR-11/FR-12). Five swim lanes — Consumer / Corporate IT, System, Logistics Driver, Recycle Plant Worker, and Government Regulator — make explicit which actor owns each step.

2. Notations Used

The table below lists every notation used in the diagram, a small snippet illustrating the symbol, its meaning, and where it appears in our specific diagram.

Symbol	Notation	Meaning	Where used in our diagram
	**Initial Node**	A solid filled black circle marking the single starting point of the activity flow.	Start of the flow, in the Consumer / Corporate IT lane, before "Enter asset description & quantity".
	**Activity Final Node**	A circle with a filled dot inside; marks where the entire activity terminates. A diagram may have more than one.	Three end points: after quote rejection, after certificate email, and after the regulator views the dashboard.
	**Activity / Action**	A rounded rectangle representing one step or action, written as an action-verb phrase.	Every processing step, e.g. "Scan barcode, mark Arrived", "Calculate price quote", "Log processing time & weight".
	**Decision Node**	A diamond with one incoming and two or more outgoing flows, each outgoing arrow labelled with a guard condition.	"Data wipe required?", "Accept price quote?" and "Verification passed?" branch points.
	**Merge Node**	A diamond that merges two or more alternate flows back into a single flow (same shape as decision, opposite direction of flows).	Where the "Yes"/"No" branches of the data-wipe check rejoin before "Fetch live metal price feed".
	**Fork / Join Node**	A thick bar splitting one flow into concurrent flows (fork) or synchronising concurrent flows back into one (join).	After "Log processing time & weight": forks into "Update status: Recycled" and "Generate Certificate of Destruction" running in parallel, then joins before emailing the certificate and updating compliance data.
	**Swim Lane (Partition)**	Vertical columns, each headed with an actor/role name; every activity is placed in the lane of the actor responsible for it.	Five lanes: Consumer / Corporate IT, System, Logistics Driver, Recycle Plant Worker, Government Regulator.
	**Control Flow (Arrow)**	A solid arrowed line showing the direction of flow between activities, decisions, and fork/join nodes.	Connects every pair of nodes in the diagram, e.g. "Arrive at pickup location" → "Inspect & photograph package".

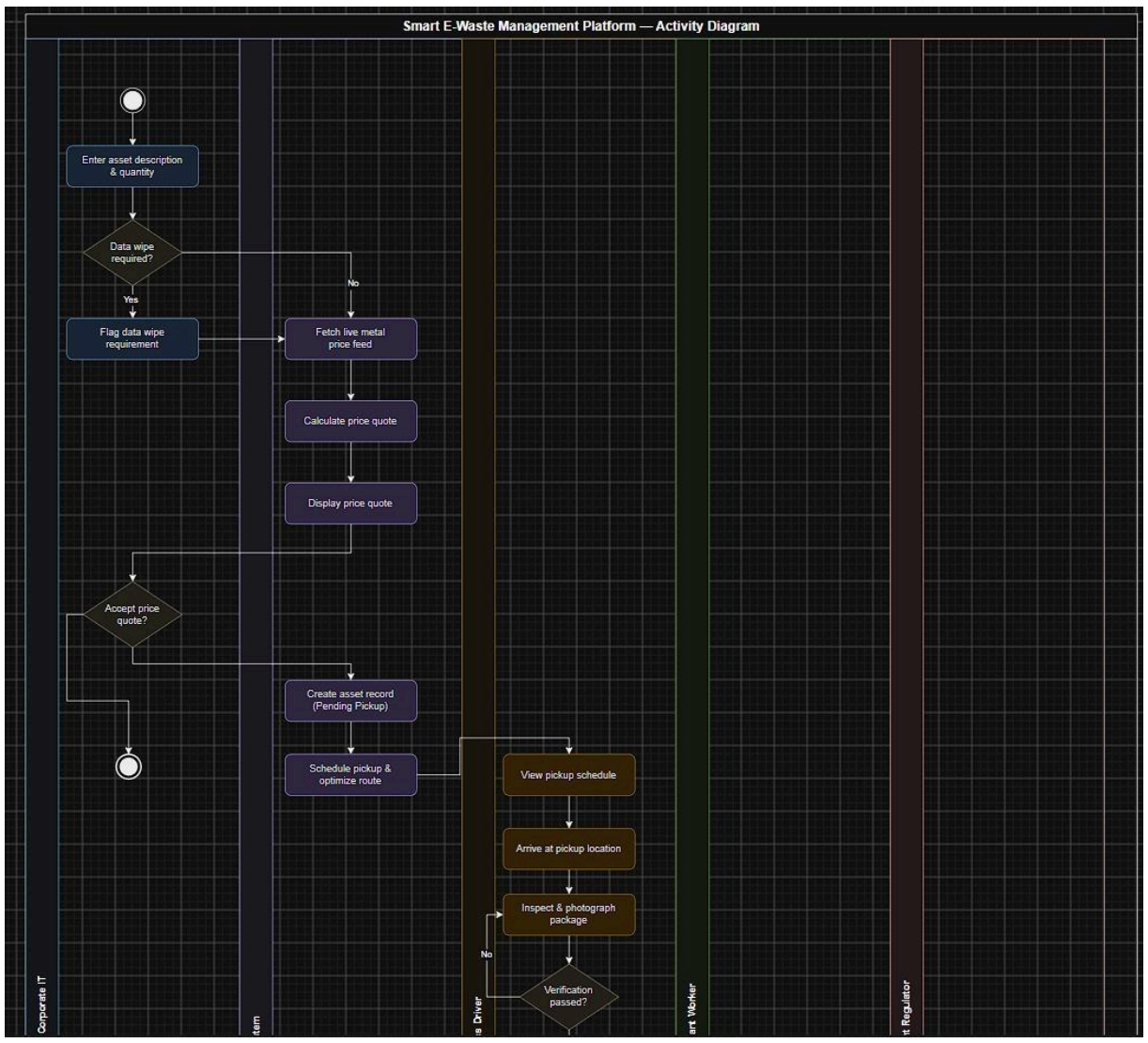
3. Advantages

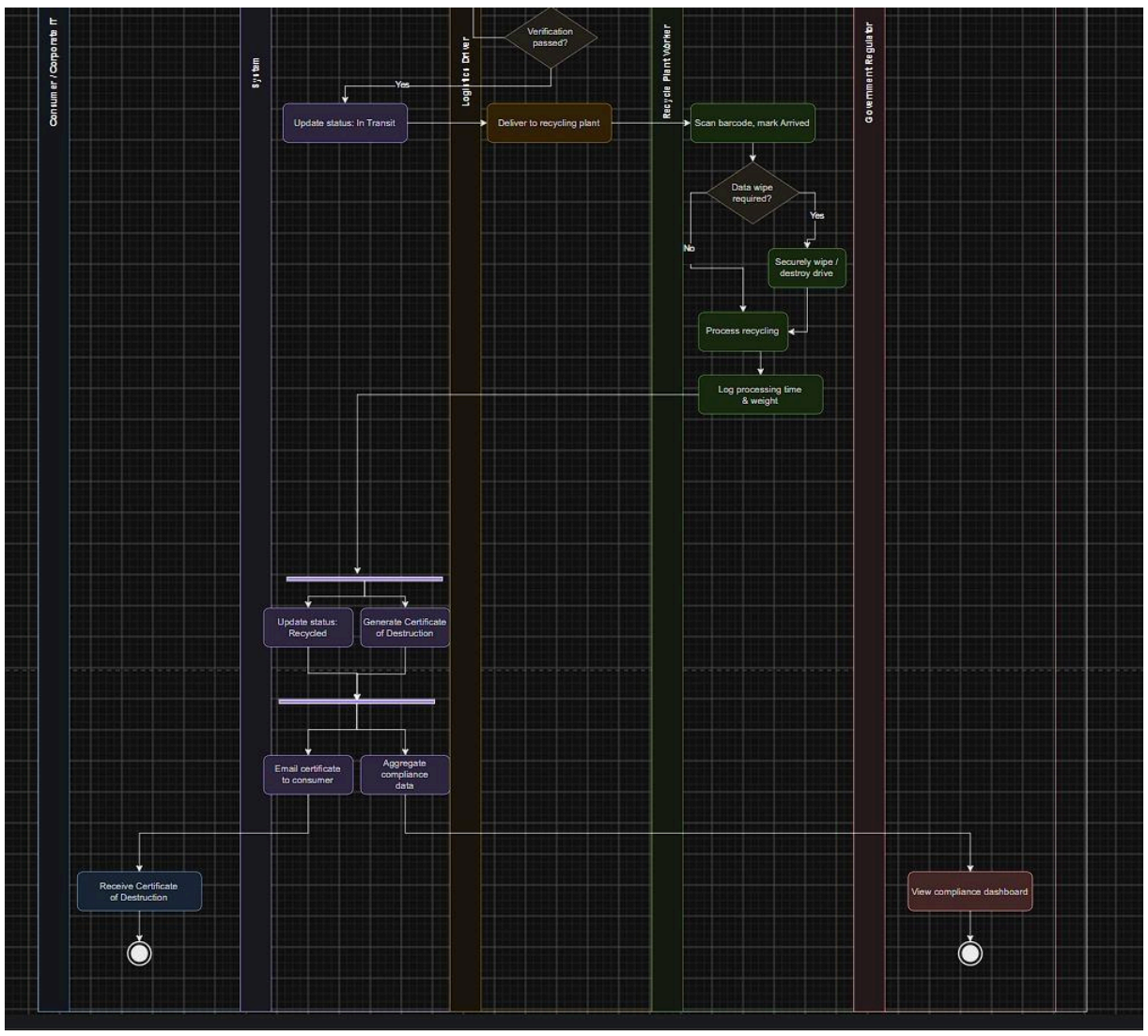
- Clearly shows the sequence and branching logic of the business process, making it easy to validate the flow against the functional requirements (FR-01 to FR-12).
- Swim lanes make responsibility for every step explicit, which is directly useful when assigning modules to the User Asset, Logistics, Recycler, and Auditor teams described in the SRS.
- The fork/join pair cleanly expresses genuine concurrency (status update and certificate generation happening at the same time), something a plain flowchart cannot represent unambiguously.
- Reads much like an ordinary flowchart, so it is easy for non-technical stakeholders — corporate IT managers, drivers, regulators — to review and sign off on.

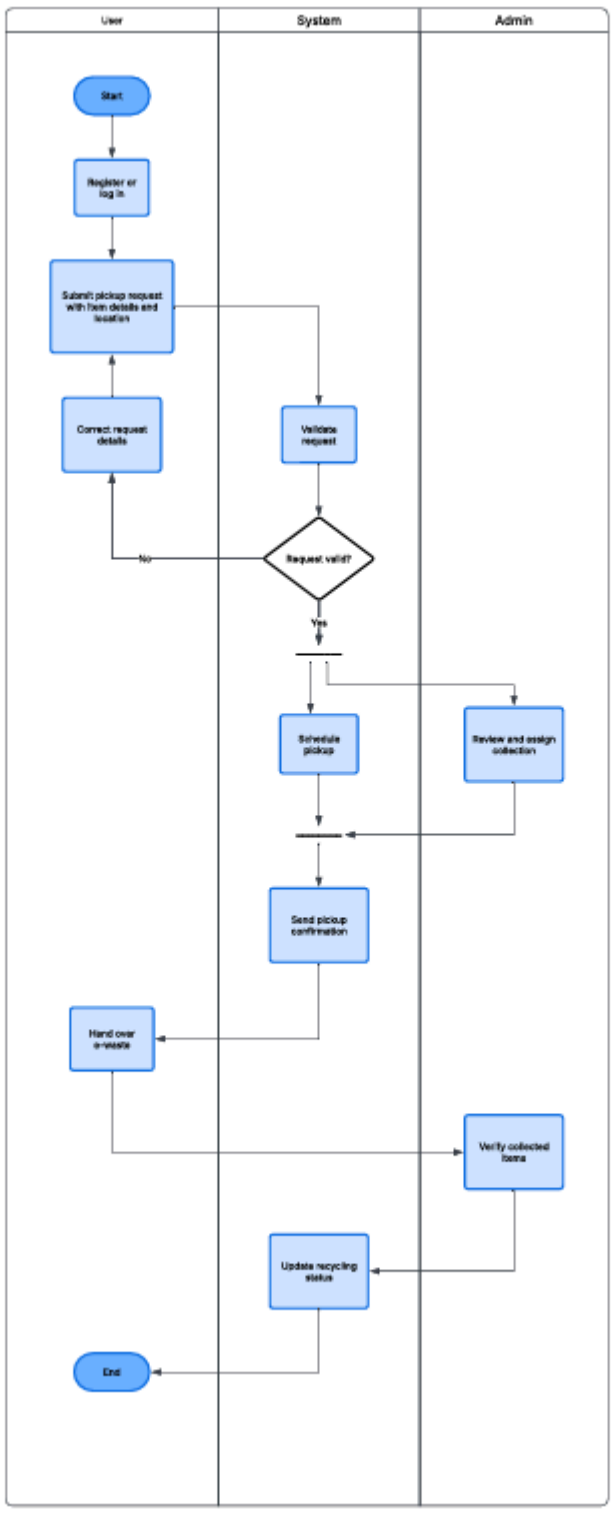
4. Disadvantages

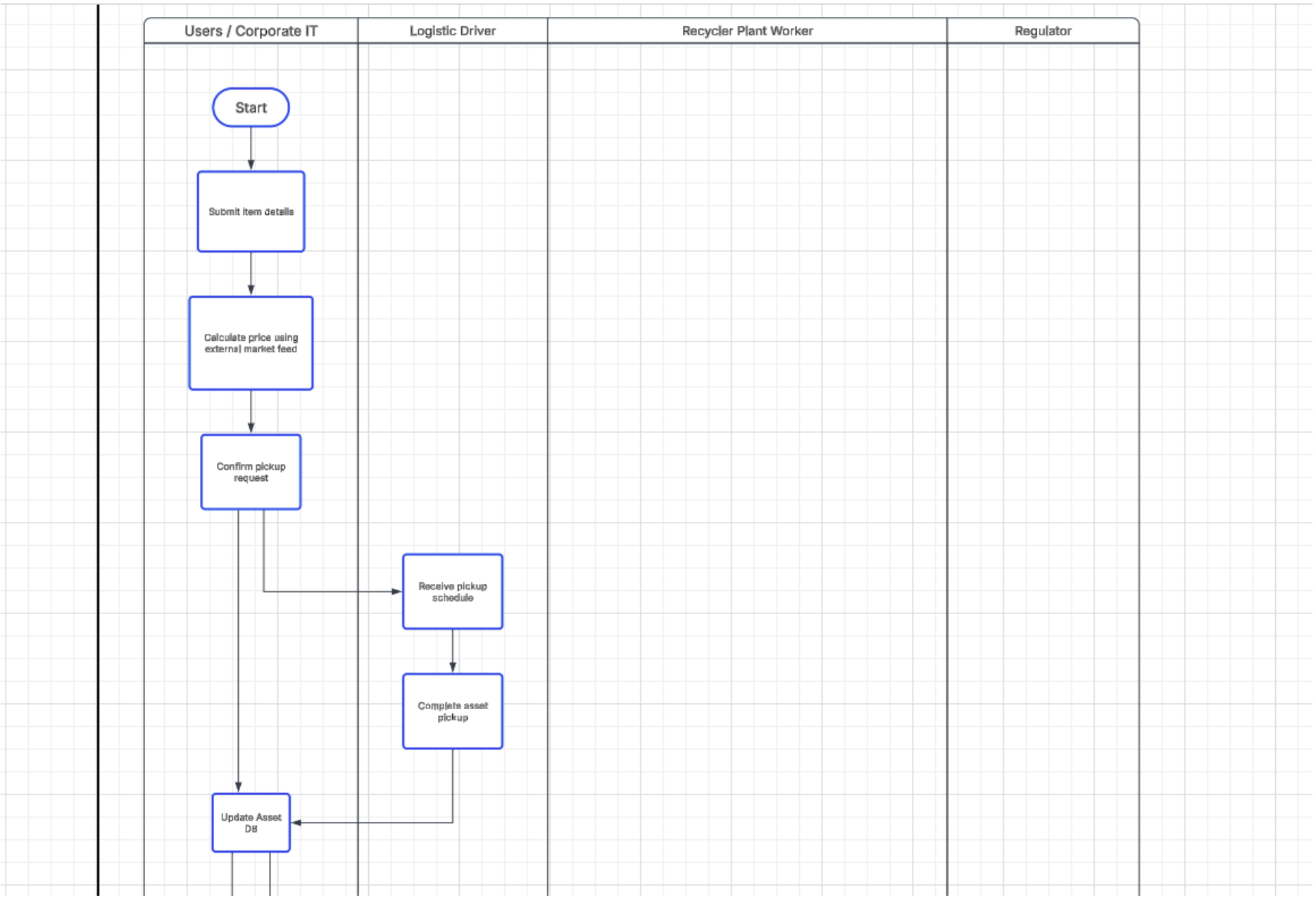
- Becomes visually dense as the number of decisions, lanes, and parallel paths grows; our diagram already needed to be split across two views to stay readable.
- Does not model the internal state or attributes of the objects being acted on (e.g. it does not show that an Asset has a status field) — that is better captured by a state or class diagram.
- Carries no timing information, so it cannot represent the platform's non-functional requirements such as the 2-second price-quote response target.
- Loop-back edges (e.g. retrying a failed packing verification) must be drawn by hand and can be misread as an error path if not labelled clearly.

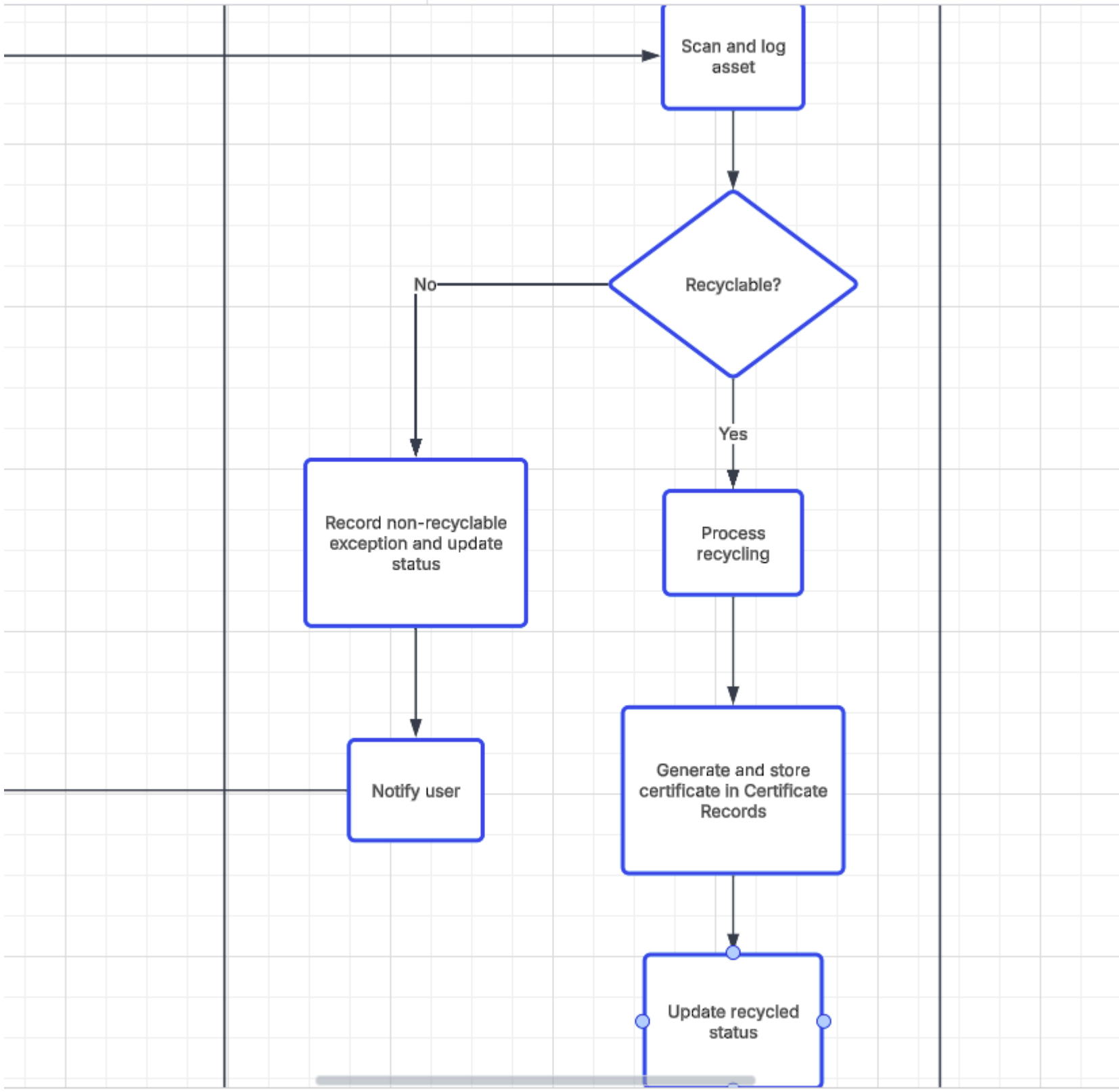
5. Output

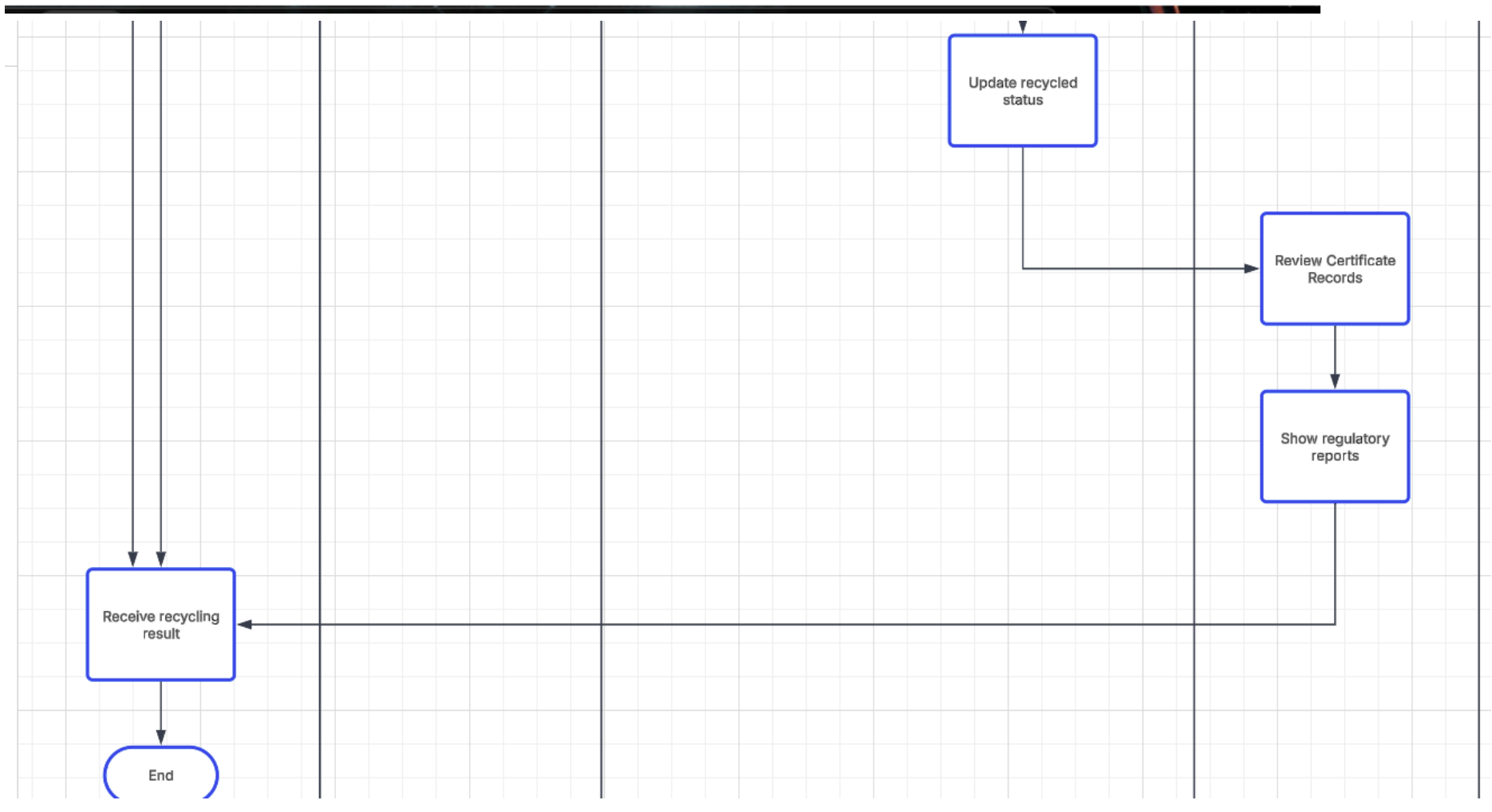

Smart E-Waste Management Platform — Activity Diagram
Enter asset description & quantity
Data wipe required?
Yes
No
Flag data wipe requirement
Fetch live metal price feed
Calculate price quote
Display price quote
Accept price quote?
Create asset record (Pending Pickup)
Schedule pickup & optimize route
View pickup schedule
Arrive at pickup location
Inspect & photograph package
No
Verification passed?
Corporate IT
tem
s Driver
art Worker
t Regulator


Consumer / Corporate IT
System
Logistics Driver
Recycle Plant Worker
Government Regulator
Verification passed?
Yes
Update status: In Transit
Deliver to recycling plant
Scan barcode, mark Arrived
Data wipe required?
Yes
No
Securely wipe / destroy drive
Process recycling
Log processing time & weight
Update status: Recycled
Generate Certificate of Destruction
Email certificate to consumer
Aggregate compliance data
Receive Certificate of Destruction
View compliance dashboard


User
System
Admin
Start
Register or log in
Submit pickup request with item details and location
Correct request details
Validate request
No
Request valid?
Yes
Schedule pickup
Review and assign collection
Send pickup confirmation
Hand over e-waste
Verify collected items
Update recycling status
End


Users / Corporate IT
Logistic Driver
Recycler Plant Worker
Regulator
Start
Submit item details
Calculate price using external market feed
Confirm pickup request
Receive pickup schedule
Complete asset pickup
Update Asset DB


Scan and log asset
Recyclable?
No
Yes
Record non-recyclable exception and update status
Process recycling
Notify user
Generate and store certificate in Certificate Records
Update recycled status


Update recycled status
Review Certificate Records
Show regulatory reports
Receive recycling result
End

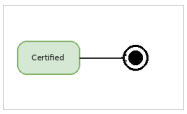
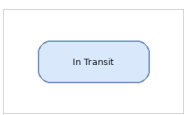
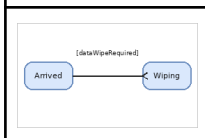
Part B — State Transition Diagram

1. Description of the Diagram

A State Transition (Statechart) Diagram is a UML behavioural diagram that models the distinct states an individual object can occupy during its lifetime, and the events, guard conditions, and actions that cause it to move between those states. Unlike an activity diagram, which is organised around actors and steps, a state diagram is organised around a single object's lifecycle.

Here, the diagram models the lifecycle of an Asset record in the Smart E-Waste Management Platform, from the moment a consumer submits it (Logged) through valuation, pickup, transit, arrival, optional data wiping, processing, and finally Recycled and Certified — or, alternatively, Cancelled if the price quote is declined. Guard conditions such as [dataWipeRequired] tie the diagram directly back to functional requirements FR-02 and FR-10 of the SRS.

2. Notations Used

Symbol	Notation	Meaning	Where used in our diagram
	**Initial Pseudostate**	A filled black circle with an outgoing arrow; marks the state an object is in immediately after creation.	Transition into the Logged (Pending Valuation) state on "submit asset details".
	**Final Pseudostate**	A circle with a ring around a filled dot (bullseye); marks that the object's lifecycle has ended.	Reached from Certified (successful path) and from Cancelled (quote-declined path).
	**State**	A rounded rectangle naming a condition the object persists in for some period, e.g. waiting for an event.	Logged, Pending Pickup, In Transit, Arrived, Data Wiping, Processing / Recycling, Recycled, Certified, Cancelled.
	**Transition**	An arrow labelled event [guard] / action, showing what triggers the move from one state to the next and what the system does when it fires.	e.g. Arrived → Data Wiping labelled [dataWipeRequired]; Recycled → Certified labelled generateCertificate() / emailCertificate().

3. Advantages

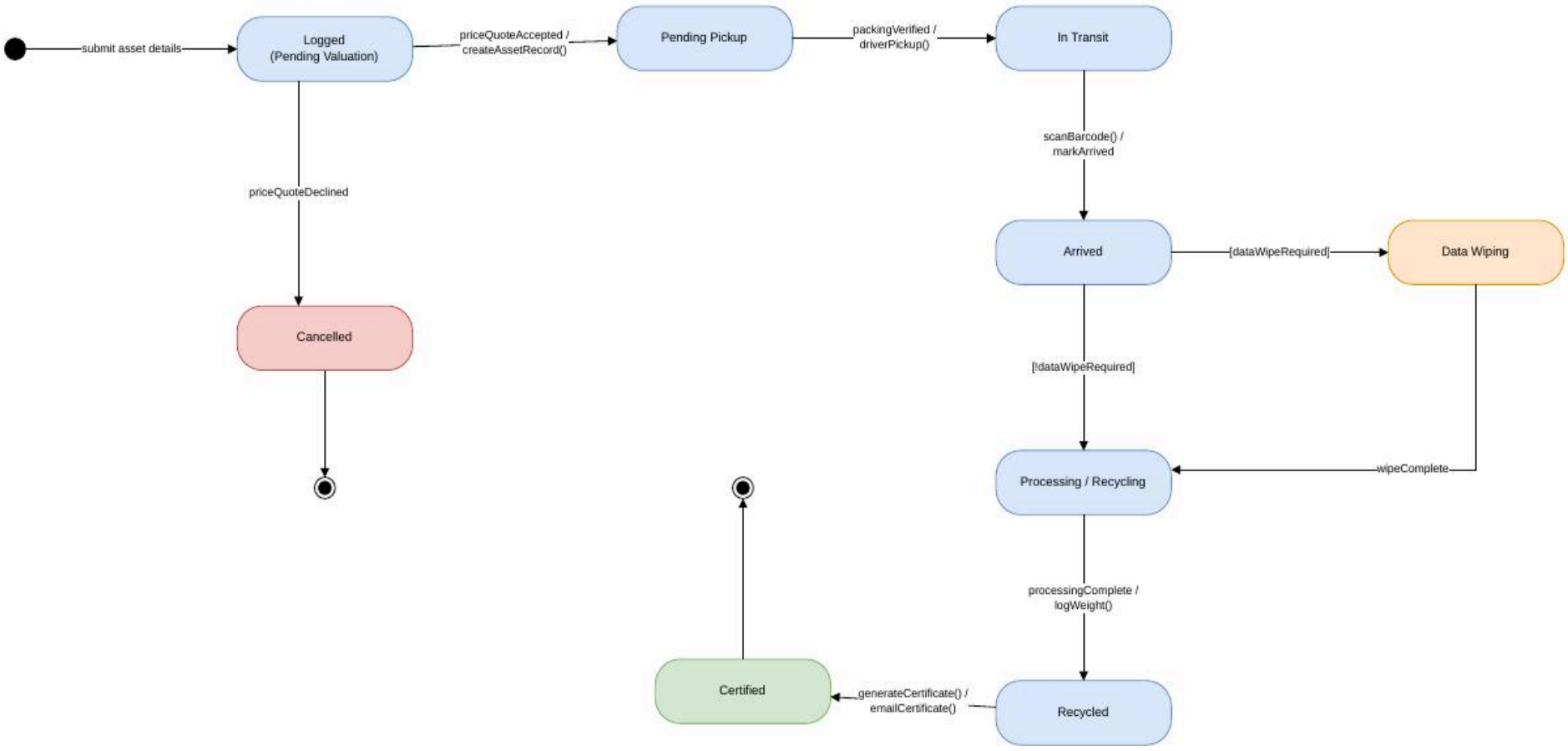
- Precisely documents every legal state an Asset can be in, which prevents invalid states — for example, an asset can never reach Certified without first passing through Recycled.
- Guard conditions make business rules explicit and directly traceable to specific functional requirements (e.g. [dataWipeRequired] to FR-02/FR-10).
- Serves as a ready reference for implementing the backend's status field and validating which transitions the API should allow.
- Compact — a single diagram documents an object's entire lifecycle without needing several separate flowcharts.

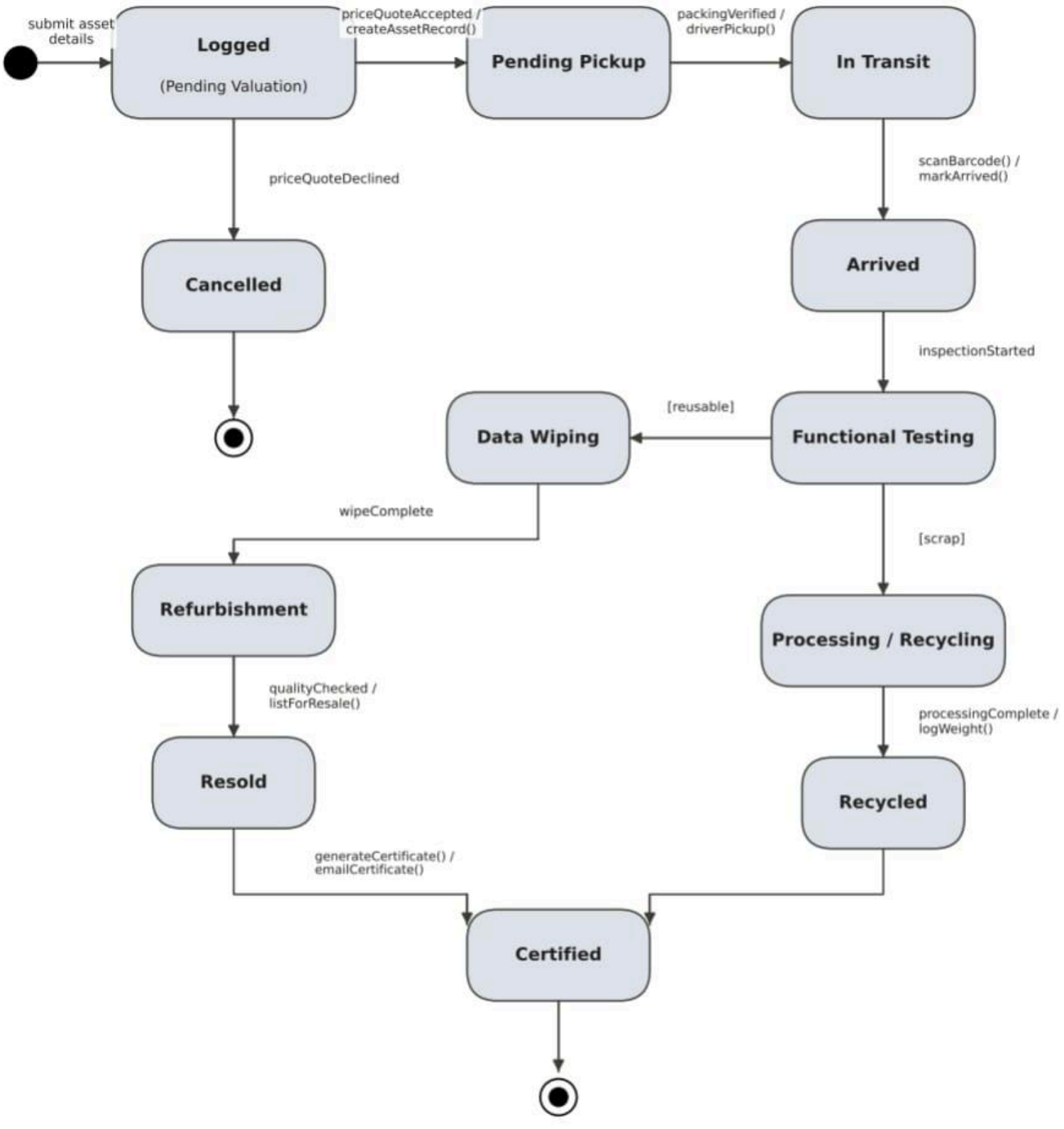
4. Disadvantages

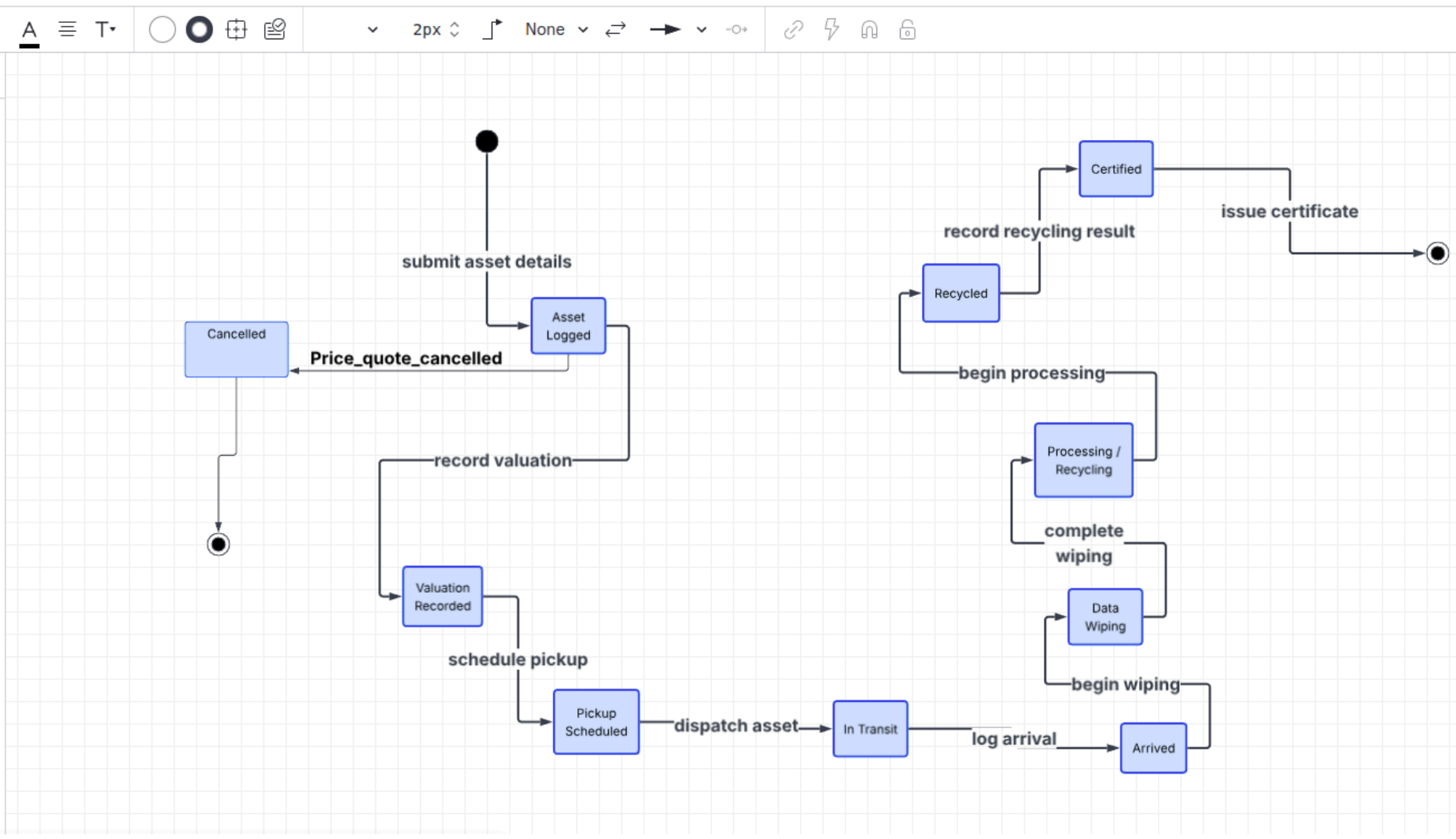
- Models only the perspective of a single object (the Asset); it says nothing about which actor performs each action, unlike the swim-laned activity diagram.
- Can suffer from state explosion if more sub-states are added later (e.g. separate sub-states for each stage of recycling), making the diagram harder to manage.
- Does not represent concurrent or parallel object states well without additional composite/orthogonal-region notation, which was not needed here but would complicate the diagram if added.

- Assumes discrete, clearly-defined events; it is a poor fit for modelling gradual or continuous processes.

5. Output




submit asset details
Logged
(Pending Valuation)
priceQuoteAccepted / createAssetRecord()
Pending Pickup
packingVerified / driverPickup()
In Transit
priceQuoteDeclined
Cancelled
scanBarcode() / markArrived()
Arrived
inspectionStarted
Functional Testing
[reusable]
Data Wiping
[scrap]
wipeComplete
Refurbishment
Processing / Recycling
qualityChecked / listForResale()
processingComplete / logWeight()
Resold
Recycled
generateCertificate() / emailCertificate()
Certified


2px
None
submit asset details
Asset Logged
Cancelled
Price_quote_cancelled
record valuation
Valuation Recorded
schedule pickup
Pickup Scheduled
dispatch asset
In Transit
log arrival
Arrived
begin wiping
Data Wiping
complete wiping
Processing / Recycling
begin processing
Recycled
record recycling result
Certified
issue certificate